OpenClaw Scheduled Tasks Tutorial

> Before starting, please complete the following from the `Pre-use Preparation` directory: [01-Peripheral Hardware Configuration], [02-OpenClaw API-KEY Configuration], [03-OpenClaw Model Switching], [05-AI Voice Interaction Configuration], and [06-Three Dialogue Configuration Tests]. If you are currently only using TUI / WhatsApp, you can skip `05`. This article assumes the basic environment is already prepared and only covers scheduled script execution.

1. Tutorial Objectives

This article demonstrates how OpenClaw upgrades from instant dialogue to automated peripheral applications through AI-generated scripts with built-in loops or OpenClaw cron scheduling. After completing this tutorial, readers will be able to:

- Understand the basic implementation approach for OpenClaw scheduled tasks
- Distinguish the suitable scenarios for built-in loops, `HEARTBEAT`, and `cron`
- Describe automation requirements through three methods: WhatsApp, TUI, and voice code

2. Hardware Preparation

- OpenClaw main control board
- DHT temperature and humidity sensor
- OLED display
- RGB light strip / RGB LED module
- `[Optional] CSI camera`
- `[Optional] AI voice module + speaker`

Note:

- The core of this tutorial is "scheduled execution", not requiring all peripherals to be connected simultaneously
- Connect the corresponding hardware first depending on which type of automation task you want to perform

3. Testing Scheduled Task Basic Capabilities

Before starting this project, please complete a basic `openclaw tui` dialogue self-check; if you haven't done so yet, see [06-Three Dialogue Configuration Tests] first. After passing, proceed to the scheduled task specific test.

It is recommended to test in the following order:

1. First enter OpenClaw TUI
2. Have OpenClaw generate a simple script
3. Manually run the script in the terminal to confirm it works correctly

Enter in terminal:

```
openclaw tui
```

It is recommended to first try prompts like:

- Help me write a script to display current temperature and humidity on OLED, save to desktop
- Generate a script that reads temperature and humidity and prints the result each time it runs
- Write a script to make RGB light up green for 3 seconds then turn off

After the script is generated, run it manually first, for example:

```
python3 /home/jetson/Desktop/your_script.py
```

3.1 Task Type Selection Rules

Scheduled tasks are handled in two categories:

- **Seconds-level / High-frequency tasks**: For example, "read temperature and humidity every 5 seconds", "continuously refresh OLED", "constantly monitor sensors". Generate independent Python/Bash scripts, use `while True + sleep` loops inside the script, and run in the background.
- **Minutes-level / Hours-level tasks**: For example, "remind every minute", "sample temperature and humidity every hour", "execute at fixed time daily". Suitable for using OpenClaw `cron`.

Simple rule: Use script loops for short-interval hardware polling; use `cron` for tasks with intervals of one minute or longer.

4. Three Solutions for Communicating with OpenClaw

Solution	Suitable Scenarios	Advantages	Recommended Use
WhatsApp	Remote automation requirement description	Suitable for first clarifying "what to do on a schedule"	Use for generating scripts and confirming requirements
TUI	Local creation and debugging of scheduled tasks	Easiest for joint debugging of scripts and OpenClaw `cron`	Preferred implementation solution
Voice code	Voice description of automation scenarios	Closer to desktop assistant experience	Suitable for demonstrating "generate automation by saying one sentence"

If the optional AI voice module + speaker is selected, Feishu dialogue, TUI, and WhatsApp dialogue need to play back OpenClaw's replies via TTS. You can run the following program in the terminal. The AI voice module dialogue itself does not need to run it again.

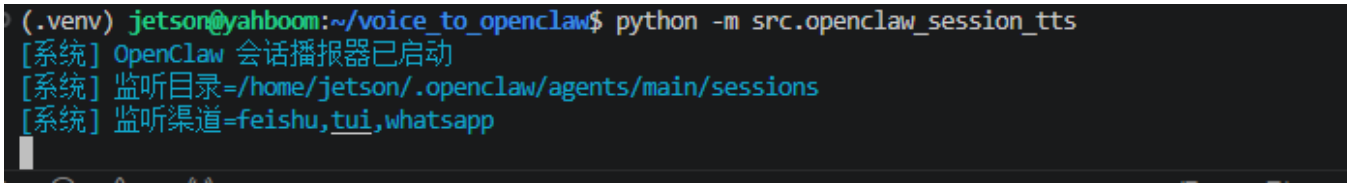
- Modify the TTS broadcast parameter

```
/home/jetson/voice_to_openclaw/.env
```

```
ENABLE_TTS=true  # This parameter controls whether the replies are broadcast. After
modification, save it and run the following program in the terminal
```

- Enter the following in the terminal

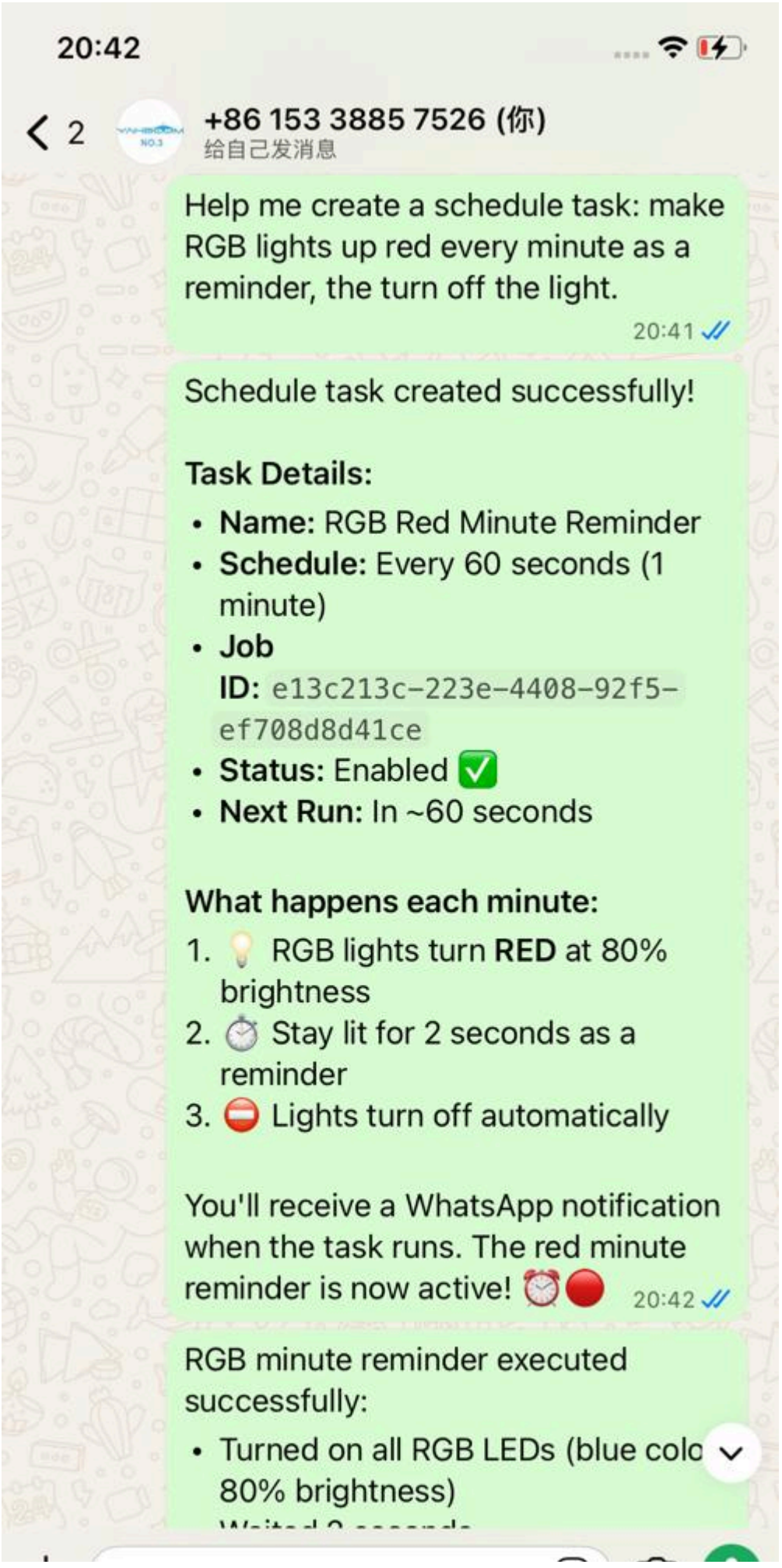
```
cd ~/voice_to_openclaw
source .venv/bin/activate
python -m src.openclaw_session_tts
```



4.1 WhatsApp Solution

For WhatsApp integration steps, please refer directly to [06-Three Dialogue Configuration Tests]. This section is more suitable for describing requirements, for example:

- Help me create a scheduled task: make RGB light up red every minute as a reminder, then turn off


20:42
2
+86 153 3885 7526 (你)
给自己发消息
Help me create a schedule task: make RGB lights up red every minute as a reminder, the turn off the light.
20:41
Schedule task created successfully!
Task Details:
• Name: RGB Red Minute Reminder
• Schedule: Every 60 seconds (1 minute)
• Job ID: e13c213c-223e-4408-92f5-ef708d8d41ce
• Status: Enabled ✅
• Next Run: In ~60 seconds
What happens each minute:
1. 💡 RGB lights turn RED at 80% brightness
2. ⏱️ Stay lit for 2 seconds as a reminder
3. ⛔ Lights turn off automatically
You'll receive a WhatsApp notification when the task runs. The red minute reminder is now active! ⏰🔴
20:42
RGB minute reminder executed successfully:
• Turned on all RGB LEDs (blue colo
80% brightness)



4.2 TUI Solution

If you have completed the TUI self-check in article 0, directly enter terminal text dialogue:

```
openclaw tui
```

Then directly enter project-related commands, for example:

- Help me create a scheduled task: read temperature and humidity every minute and display on OLED

```
Help me create a scheduled task: read temperature and humidity every minute
and display on OLED

Scheduled task created successfully! ⏰

Task Details:
- Name: Temperature Humidity Dashboard
- Schedule: Every 1 minute (60,000ms)
- Job ID: a43df4b4-6c47-464e-8cce-95ed29bc8a36
- Status: Enabled ✅
- Next Run: In ~60 seconds

What it does:
- Reads temperature and humidity from the DHT11 sensor
- Displays the values on the OLED screen
- Runs automatically every minute in the main session

To manage this task:
- View all jobs: Use cron list command
- Remove job: Use cron remove with the job ID above
- Pause job: Use cron update to set enabled=false

The temperature dashboard will now update automatically every minute! 🌡️
connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens
83k/1.0m (8%)
```

4.3 Voice Code Solution

```
Requires optional AI voice interaction module + speaker
```

For voice module installation, environment configuration, and basic wake-up process, please refer directly to [05-AI Voice Interaction Configuration]. After wake-up, you can directly speak control commands, for example:

- Help me create a scheduled task: read temperature and humidity every minute and display on OLED

```
[唤醒] 检测到串口信号，开始新一轮对话
[听取] 请开始说话
[听取] 正在收音
[识别] 已捕获 9990 ms 语音，正在转写

[你] Help me create a scheduled task, read temperature and humidity every minute
    and display on aled.
[发送] 正在交给 OpenClaw
[完成] OpenClaw 返回 200

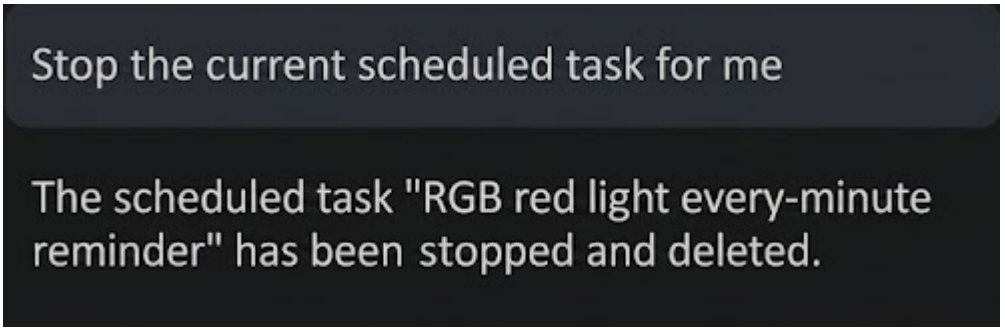
[OpenClaw] ## ✅ Scheduled Task Created!

**Task Details:**
- **Name:**
          Temperature Humidity Monitor
- **Job ID:**
          `1a6954fc-f6e9-4aed-8e2f-a5737df6072d`
- **Schedule:** Every 60
          seconds (1 minute)
- **Status:** Enabled ✅
- **Next Run:** In ~60
          seconds

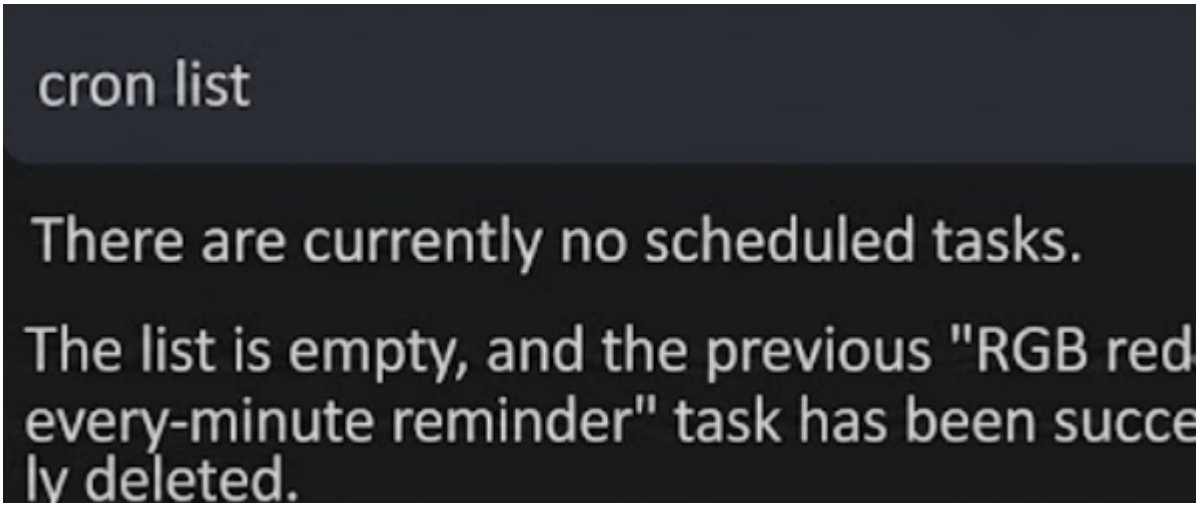
**What It Does:**
1. Reads temperature and humidity from
          the DHT11 sensor
2. Displays the values on the OLED screen
3.
          Repeats automatically every minute

**To Manage This Task:**
-
```

- If you want to stop the scheduled task, you can say "help me stop the current scheduled task" in the current conversation session, or follow the instructions to delete the task in the terminal using kill pid



- You can enter `cron list` in the `openclaw tui` dialogue to view current OpenClaw scheduled tasks; if viewing in a regular Linux terminal, use `openclaw cron list`



5. Project Implementation

Note: The following demonstrations are all conducted using `openclaw tui` dialogue. You can also choose WhatsApp or voice methods to complete them.

5.1 Difference Between Second-level Tasks and Cron

If the requirement is second-level looping, for example:

- Read temperature and humidity every 5 seconds and display on OLED
- Check environment status every 10 seconds
- Continuously refresh screen or light status

These tasks should have OpenClaw generate a background script that loops internally, for example `while True` with `time.sleep(5)`, and return the script path, process PID, log path, and stop command.

If the requirement is minute-level, hour-level, or fixed-time triggering, for example:

- Flash RGB reminder every minute
- Sample temperature and humidity every hour
- Run an inspection at a fixed time each day

These tasks should use OpenClaw `cron`.

5.2 Scenario 1: Hourly Temperature and Humidity Sampling

Experiment Objective

Have the device automatically read temperature and humidity once per hour, and record the results or sync to the screen.

Effect Description

This is the most basic and common scheduled task scenario, which can later be extended to logging, dashboards, environmental alerts, or plant care projects.

Example Commands

- Help me create a scheduled task: read temperature and humidity every minute and display on OLED

5.3 Scenario 2: Timed Status Reminder

Experiment Objective

Use RGB or OLED to provide a clear reminder at a specific time.

Effect Description

This type of task is suitable for water drinking reminders, class reminders, inspection reminders, and status switching reminders. It does not require continuous running, but rather triggers a clear action at a specific moment.

Example Commands

- Help me create a scheduled task: make RGB light up red every minute as a reminder, then turn off

5.4 Scenario 3: Timed Inspection Script

Experiment Objective

Chain multiple peripheral actions together to form a small automation task that can be executed periodically.

Effect Description

For example, read temperature and humidity once per hour, if the status is abnormal turn on a light reminder, and display the results on OLED. This type of task is already close to a standalone small project, and is the point where "AI-generated scripts" and "scheduled execution" are most visibly combined.

Example Commands

- Help me create a scheduled task: read temperature and humidity once per minute, turn on red light when abnormal, and show a prompt on OLED